



Project Progress Report

RESTAURANT MANAGEMENT WEB APPLICATION

Framework	Django (Python 3.x)
Database	MySQL Restaurant DB
Architecture	MVT (Model-View-Template)
Modules	7 functional modules
Team Size	4 students
GitHub link	https://github.com/pettangroddysenpai-ship-it/Restaurant-Management-App

Group Members	Participation
Pettang Feumba Roddy Alann	20%
Tiofack Kaze Franck	20%
Matchim Dzuyim Verenne Celia	20%
Mekolle Tuatsop Dalhia Aure	20%
Messi Zang Paul Jacques	20%

LECTURER: MR. MBIETEU AMOS MB.

DATE: FRIDAY, 7TH MARCH 2026

Table of Contents

1. Introduction	1
2. Phase 1 project setup and configuration (Necessary tools for Django, SQL and Python Tasks)	2
2.1 Download and Installation of Git, GitHub and Creation of Repository	
2.2 Download and Installation of SQL and Django tools	
2.3 Django Project Setup	
Project Management	3
6.1 Gantt Chart	
6.2 Class Diagram	
2. Phase 2 Django Models (23 SQL Tables in Python)	4
2.1 Orders App Models	
2.2 Products App Models	
2.3 Inventory, HR, Delivery & Finance Models	
3. Phase 3 Authentication & User Roles	6
3.1 Custom User Model	
3.2 First Migration & Superuser	
3.3 Login / Logout Views	
3.4 Role-Based Permission Decorators	
4. Phase 4 Templates & Base Interface	8
4.1 Static Files & Template Folder Structure	
4.2 Base Template (base.html)	
4.3 Child Template Pattern	
5. Phase 5 Views, URLs & Business Logic	10
5.1 Standard CRUD Pattern	
5.2 URL Wiring	
5.3 Low-Stock Alert Logic	
5.4 Invoice Generation	
7. Challenges & Solutions	14
8. Conclusion	15

1. Introduction

As part of the Python Programming course delivered at the PKFIE Institute of Excellence, our group of four (4) students was tasked with designing and developing a comprehensive restaurant management web application. This application is built on the Django (Python) framework and uses a pre-designed MySQL database composed of 23 tables covering all the operational needs of a modern restaurant.

In order to efficiently organize our work and meet the project requirements, we divided the development of the application into three (3) distinct phases as follows:

Phase	Title	Description
Phase 1	Setup & Configuration	Setting up the development environment, installing all required tools, creating the Django project, configuring MySQL and importing the existing database.
Phase 2	Models & Application Foundation	Writing Django models corresponding to the 23 SQL tables, setting up the authentication system and user role management.
Phase 3	Interfaces, Business Logic & Deployment	Building views, HTML templates, the dashboard, financial reports, implementing security measures and preparing all final deliverables.

This report documents the progress made during Phase 1, which forms the technical foundation of the entire application. Each step is presented with the commands used and a screenshot to support the work done.

This report documents the development of a Restaurant Management Web Application built using the Django framework (Python 3.x) and a MySQL relational database comprising 23 tables. The project was conceived to digitalize and centralize the core operations of a restaurant, including order management, inventory tracking, human resources, delivery coordination, and financial reporting.

The application follows the Model-View-Template (MVT) architectural pattern native to Django, ensuring a clean separation of concerns between data models, business logic, and presentation. Seven functional modules were implemented to cover the full operational scope of a modern restaurant.

This report focuses specifically on Phases 2 through 5 of the development lifecycle, covering the following key milestones:

- Phase 2 Defining all Django models that map to the 23 existing SQL tables.
- Phase 3 Implementing authentication, custom user roles, and role-based access control.
- Phase 4 Building the shared HTML template system and the base interface layout.
- Phase 5 Developing the views, URL routing, and core business logic for each module.

In addition, this report includes a Project Management section presenting the Gantt chart and class diagrams used to plan and design the system. A dedicated section on challenges encountered during development and the solutions applied is also provided, followed by a conclusion summarizing outcomes and lessons learned.

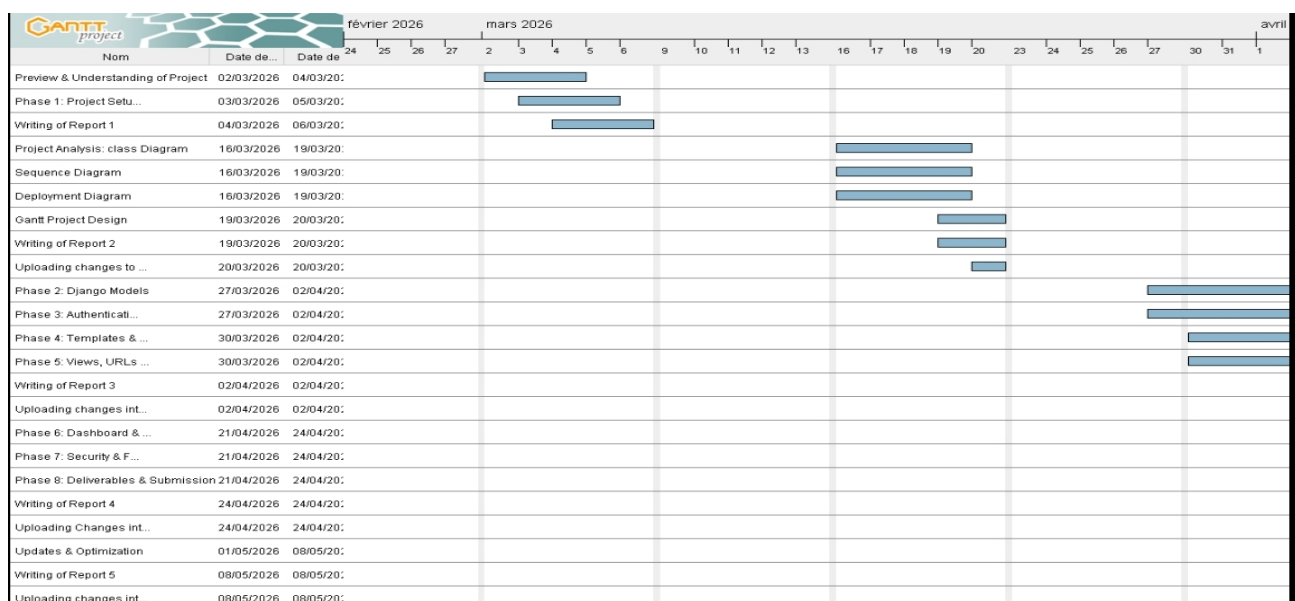
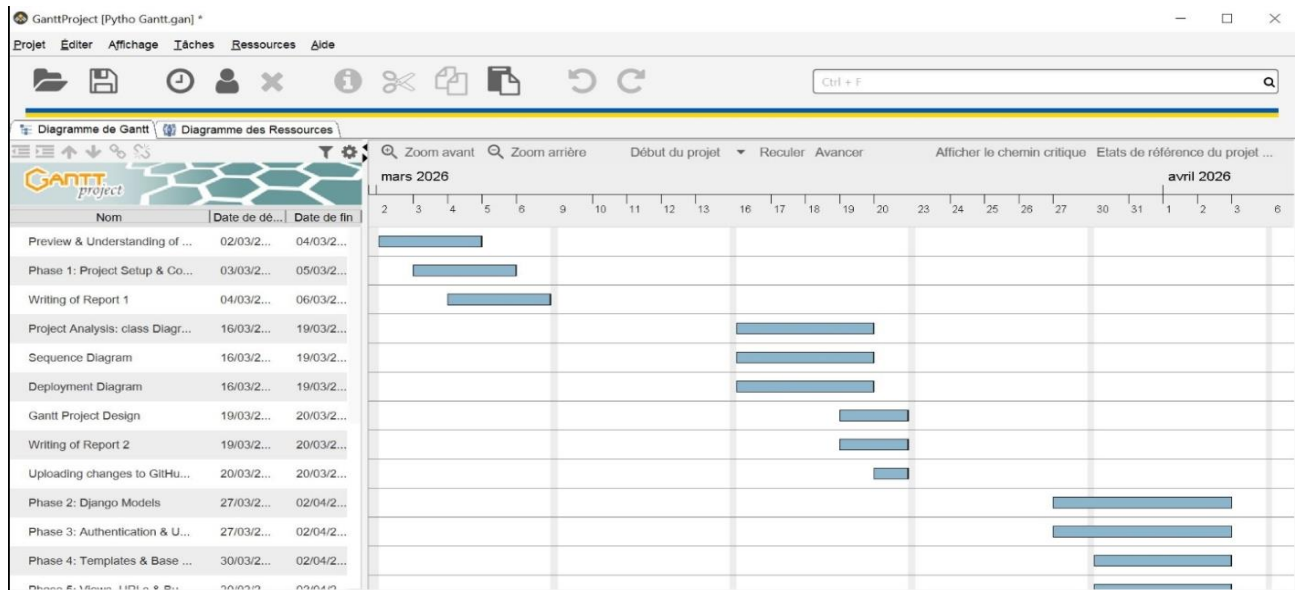
The development of a restaurant management system requires proper planning, analysis, and design in order to ensure efficiency and reliability. This project focuses on

modeling and organizing the different activities involved in managing a restaurant through the use of software engineering tools.

Effective project management was essential to coordinate the development of seven modules across multiple phases. Two key planning and design tools were used: a Gantt chart to schedule and track progress, and class diagrams to model the system's data structure and relationships.

6.1 Gantt Chart

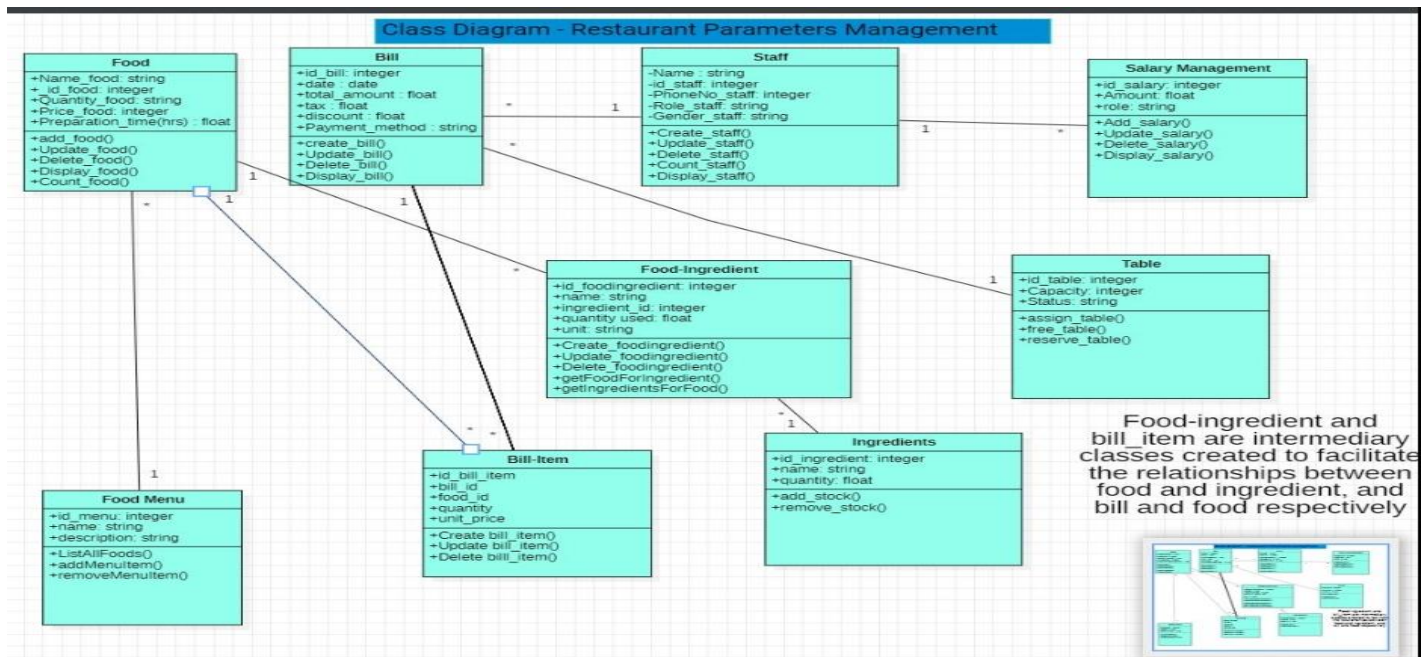
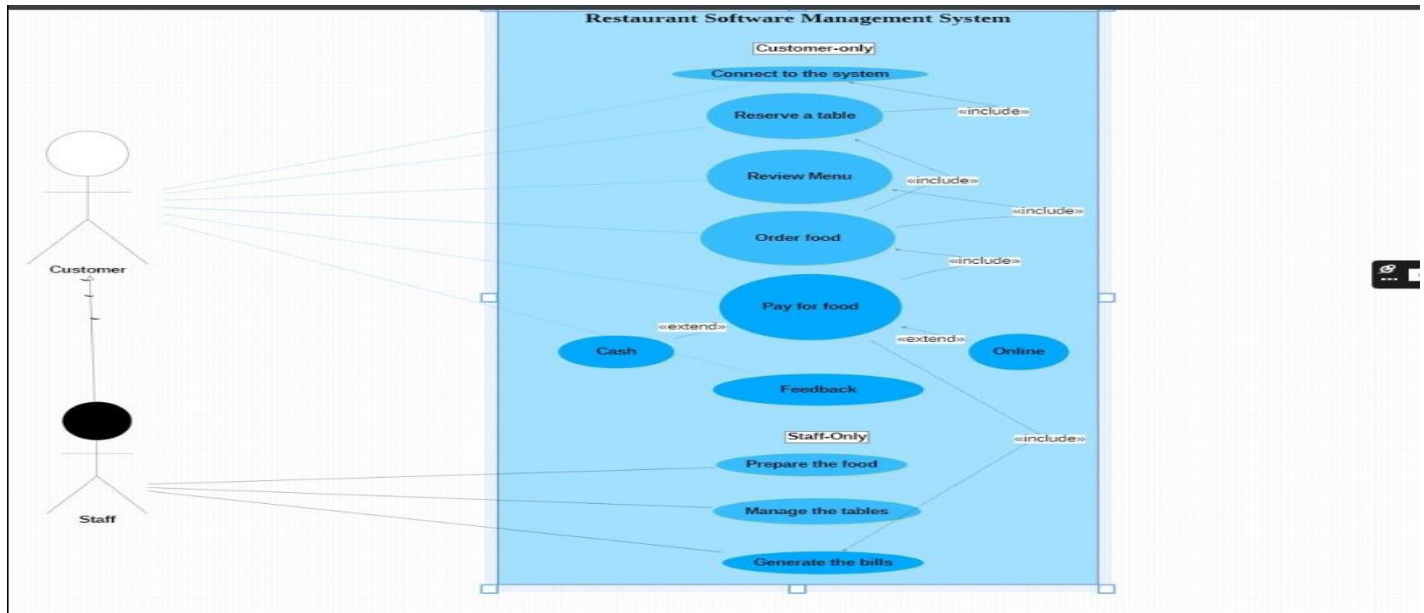
The Gantt chart below illustrates the planned timeline for the project across all development phases. Each phase is represented as a horizontal bar spanning its estimated duration, allowing the team to visualize task dependencies, parallelization opportunities, and overall project progress.



6.2 Class Diagram

The class diagram below represents the object-oriented structure of the Django application, showing all model classes, their attributes, data types, and the relationships between them (one-to-many, many-to-many, and foreign key associations).

Key relationships visible in the diagram include: Commande linked to Client and LigneCommande; LigneCommande linked to Produit; CompositionProduit linking Produit and Ingredient; Stock linked to Ingredient; and Affectation linking Employe and Poste.



2. Phase 1: Project Setup & Configuration

The goal of Phase 1 is to prepare the entire Django project structure before writing any application code. This phase covers seven (7) essential steps that were carried out in the order listed below.

Step 1: Installing the Required Tools

The first action was to install all the Python libraries needed for Django to function and to connect to MySQL. These installations were carried out using the pip tool in the Windows Command Prompt.

The following packages were installed:

- **Django**; the main Django web framework
- **MySQL client**; the connector that allows Django to communicate with MySQL
- **pillow** ; image handling Library
- **python-decouple**; stores sensitive credentials (passwords) in a separate .env file away from GitHub

Step 2: Creating the Django Project

After installing the required tools, we created the base Django project structure. This was done from the terminal inside our group's working folder.

```
django-admin startproject restaurant_project
cd restaurant_project
```

These commands automatically generate the following file structure:

```
restaurant_project/
├── manage.py           ← Django entry point
└── restaurant_project/
    ├── settings.py     ← main configuration file
    ├── urls.py         ← main URL router
    └── wsgi.py
```

Step 3: Creating One Django App Per Module

In line with the functional requirements of the project, each restaurant management module was translated into an independent Django application. This modular organization respects the MVT (Model-View-Template) architecture required by the project specifications.

The following eight (8) applications were created using the commands below and each command creates a folder containing the following base files (example with the orders app):

Application	Module Covered	SQL Tables Used
users	User Management	CustomUser (new table)
products	Product & Recipe Mgmt	Produit, Ingredient, Composition_Produit

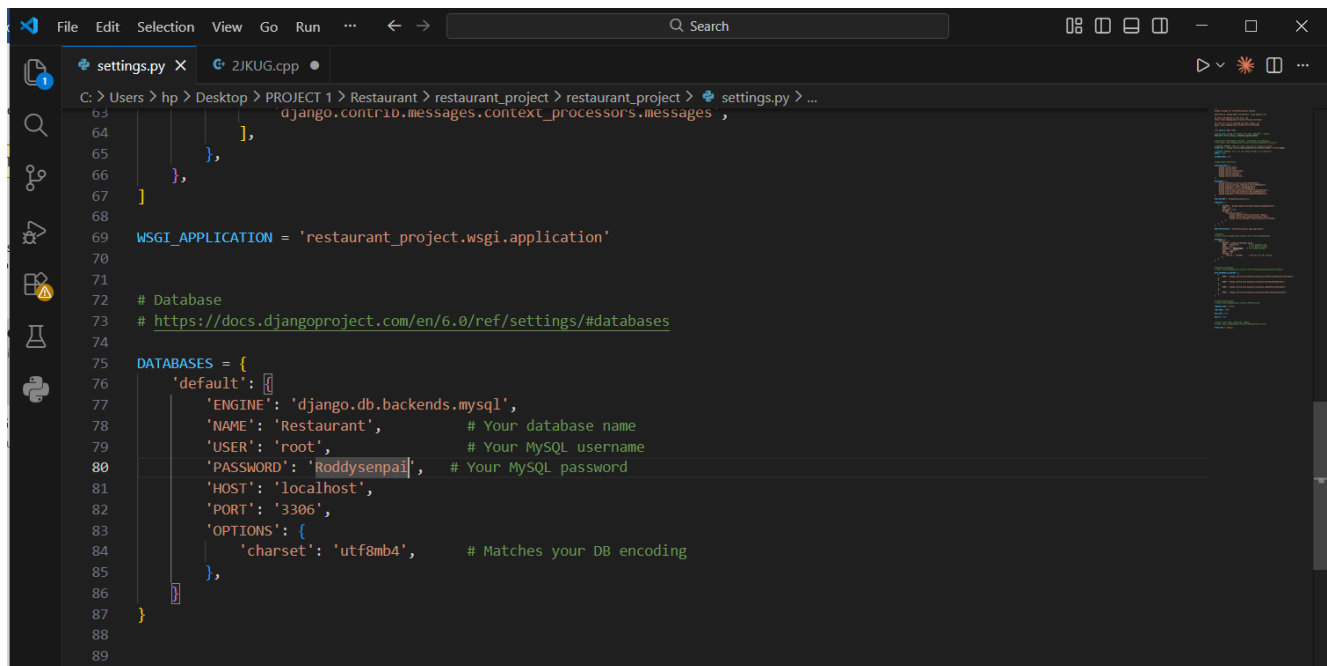
orders	Order Management	Commande, Ligne_Commande, Client, Table, Reservation
inventory	Inventory Management	Stock, Variation_Stock, Approvisionnement, Fournisseur
hr	HR Management	Employe, Poste, Affectation, Supervision
delivery	Delivery Management	Livraison, Deplacement, Vehicule
Finance	Finance Management	Facture, Equipement, Maintenance, Action_Marketing
dashboard	Dashboard & Reports	Aggregation of all tables

The screenshot shows a Windows File Explorer window titled 'Restaurant-Management-App'. The address bar shows the path 'PROJECT 1 > Restaurant-Management-App'. The left sidebar shows the 'Accès rapide' (Quick access) section with various locations like 'Bureau', 'Téléchargements', 'Documents', 'Images', 'CHEMISTRY', 'LEADERSHIP', 'Python', 'restaurant_project', 'OneDrive', 'Ce PC', and 'Réseau'. The main pane displays a list of files and folders with columns for 'Nom', 'Modifié le', 'Type', and 'Taille'. The 'orders' folder is highlighted.

Nom	Modifié le	Type	Taille
dashboard	06/03/2026 10:35	Dossier de fichiers	
delivery	06/03/2026 10:35	Dossier de fichiers	
finance	06/03/2026 10:35	Dossier de fichiers	
hr	06/03/2026 10:34	Dossier de fichiers	
inventory	06/03/2026 10:34	Dossier de fichiers	
orders	06/03/2026 10:34	Dossier de fichiers	
products	06/03/2026 10:33	Dossier de fichiers	
restaurant_project	06/03/2026 10:33	Dossier de fichiers	
users	06/03/2026 10:33	Dossier de fichiers	
.gitignore	06/03/2026 14:01	Document texte	5 Ko
manage	06/03/2026 10:30	Fichier source Pyth...	1 Ko
README	06/03/2026 14:01	Fichier source Mar...	1 Ko

Step 4: Configuring MySQL in settings.py

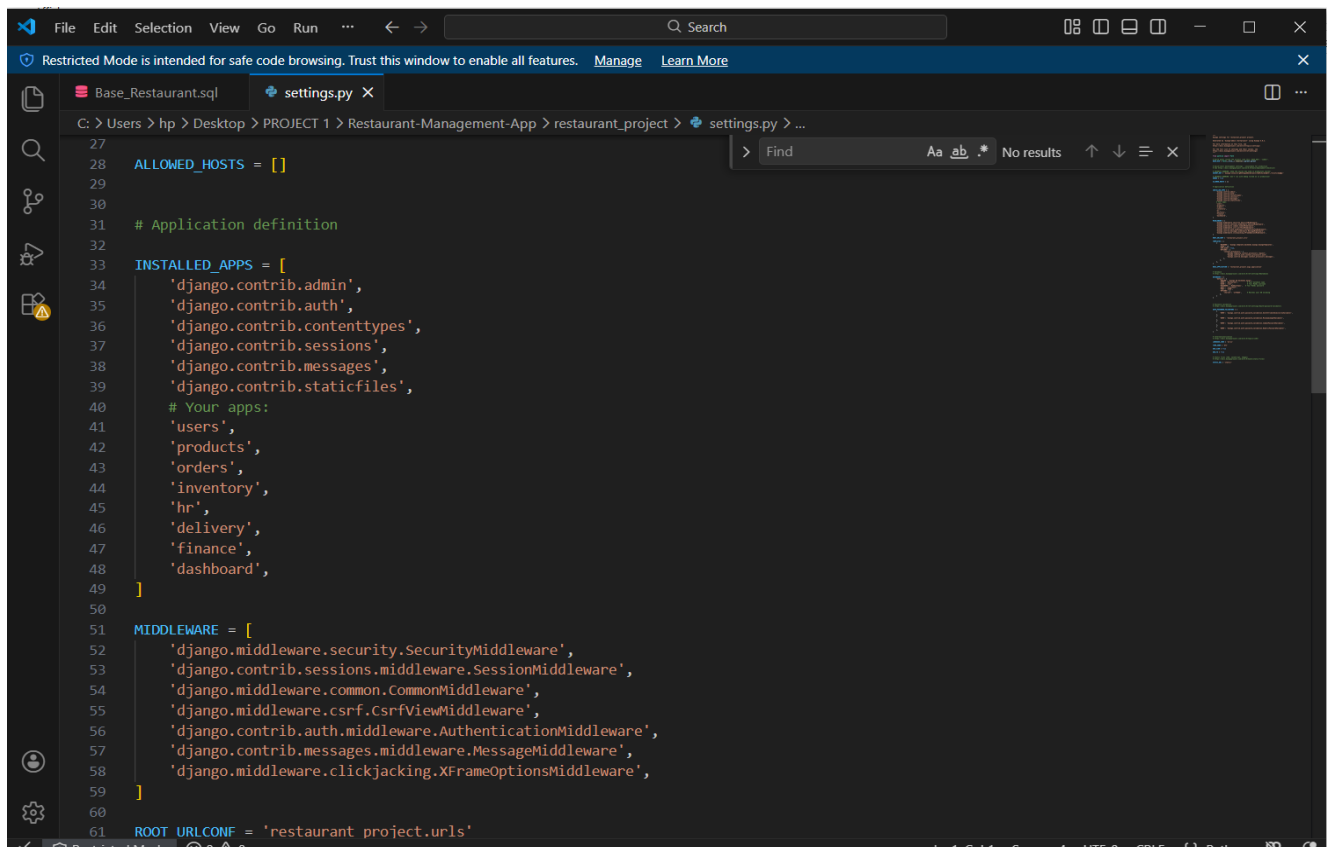
We then configured the connection between Django and the MySQL database. To do this, the DATABASES block in the settings.py file was updated to point to the Restaurant database, using the standard port 3306 and the utf8mb4 encoding that matches the existing database.



```
63     django.contrib.messages.context_processors.messages ,
64 ],
65 },
66 },
67 ],
68
69 WSGI_APPLICATION = 'restaurant_project.wsgi.application'
70
71
72 # Database
73 # https://docs.djangoproject.com/en/6.0/ref/settings/#databases
74
75 DATABASES = {
76     'default': {
77         'ENGINE': 'django.db.backends.mysql',
78         'NAME': 'Restaurant',          # Your database name
79         'USER': 'root',               # Your MySQL username
80         'PASSWORD': 'Roddysenpai',   # Your MySQL password
81         'HOST': 'localhost',
82         'PORT': '3306',
83         'OPTIONS': {
84             'charset': 'utf8mb4',     # Matches your DB encoding
85         },
86     },
87 }
88
89
```

Step 5: Registering the Apps in settings.py

For Django to recognise the eight applications created in Step 3, each one was added to the `INSTALLED_APPS` list in `settings.py`. Without this step, Django would completely ignore those modules.



```
27
28 ALLOWED_HOSTS = []
29
30
31 # Application definition
32
33 INSTALLED_APPS = [
34     'django.contrib.admin',
35     'django.contrib.auth',
36     'django.contrib.contenttypes',
37     'django.contrib.sessions',
38     'django.contrib.messages',
39     'django.contrib.staticfiles',
40     # Your apps:
41     'users',
42     'products',
43     'orders',
44     'inventory',
45     'hr',
46     'delivery',
47     'finance',
48     'dashboard',
49 ]
50
51 MIDDLEWARE = [
52     'django.middleware.security.SecurityMiddleware',
53     'django.contrib.sessions.middleware.SessionMiddleware',
54     'django.middleware.common.CommonMiddleware',
55     'django.middleware.csrf.CsrfViewMiddleware',
56     'django.contrib.auth.middleware.AuthenticationMiddleware',
57     'django.contrib.messages.middleware.MessageMiddleware',
58     'django.middleware.clickjacking.XFrameOptionsMiddleware',
59 ]
60
61 ROOT_URLCONF = 'restaurant_project.urls'
```

Step 6: Importing the MySQL Database

The database file provided for the project (Base_Restaurant__1_.sql) already contains all 23 tables as well as test data. This database was imported directly into MySQL via the Windows Command Prompt.

6.1 Installing MySQL

Before importing, it was necessary to install MySQL Server on our Windows machine. We downloaded the installer from the official website dev.mysql.com/downloads/installer, selected the Custom installation type to limit data usage, and configured the server with a root password.

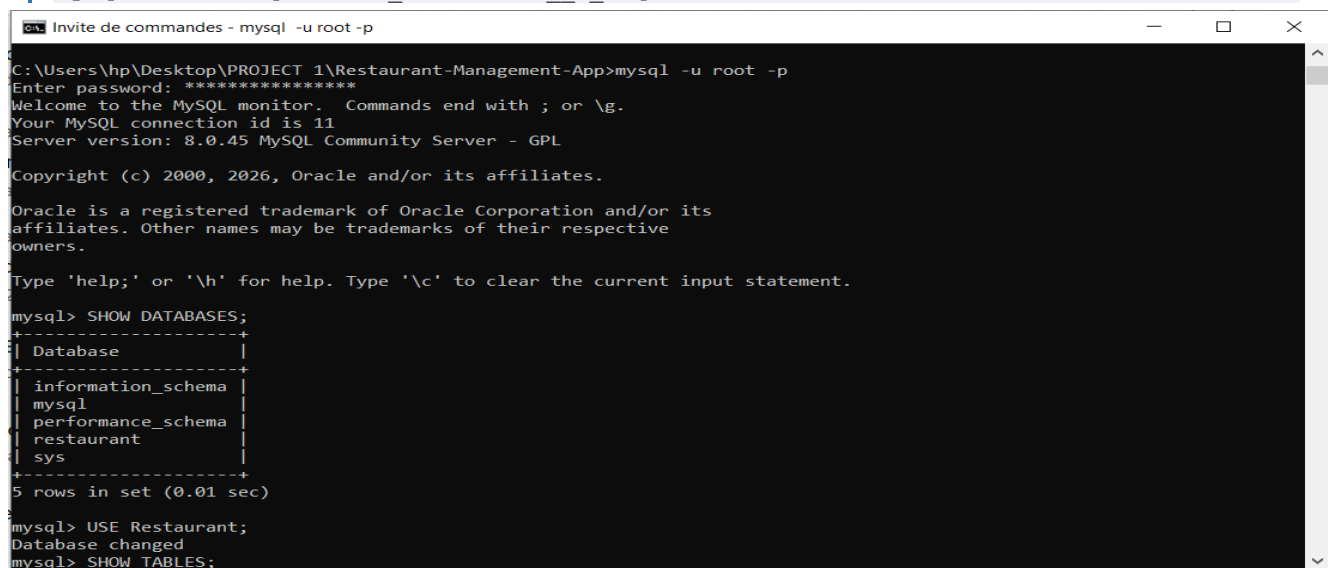
After installation, the path to the MySQL executables was added to the Windows system environment variables (PATH) so that the mysql command could be used from any terminal:

```
C:\Program Files\MySQL\MySQL Server 8.0\bin
```

6.2 Importing the SQL File

Once MySQL was correctly installed and accessible from the terminal, we navigated to the folder containing the SQL file and ran the import command:

```
cd "C:\Users\hp\Desktop\PKFOKAM\FRESHMAN 1\Python"
mysql -u root -p < Base_Restaurant__1_.sql
```



```
CA Invite de commandes - mysql -u root -p
C:\Users\hp\Desktop\PROJECT 1\Restaurant-Management-App>mysql -u root -p
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.45 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SHOW DATABASES;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| restaurant |
| sys |
+-----+
5 rows in set (0.01 sec)

mysql> USE Restaurant;
Database changed
mysql> SHOW TABLES;
```

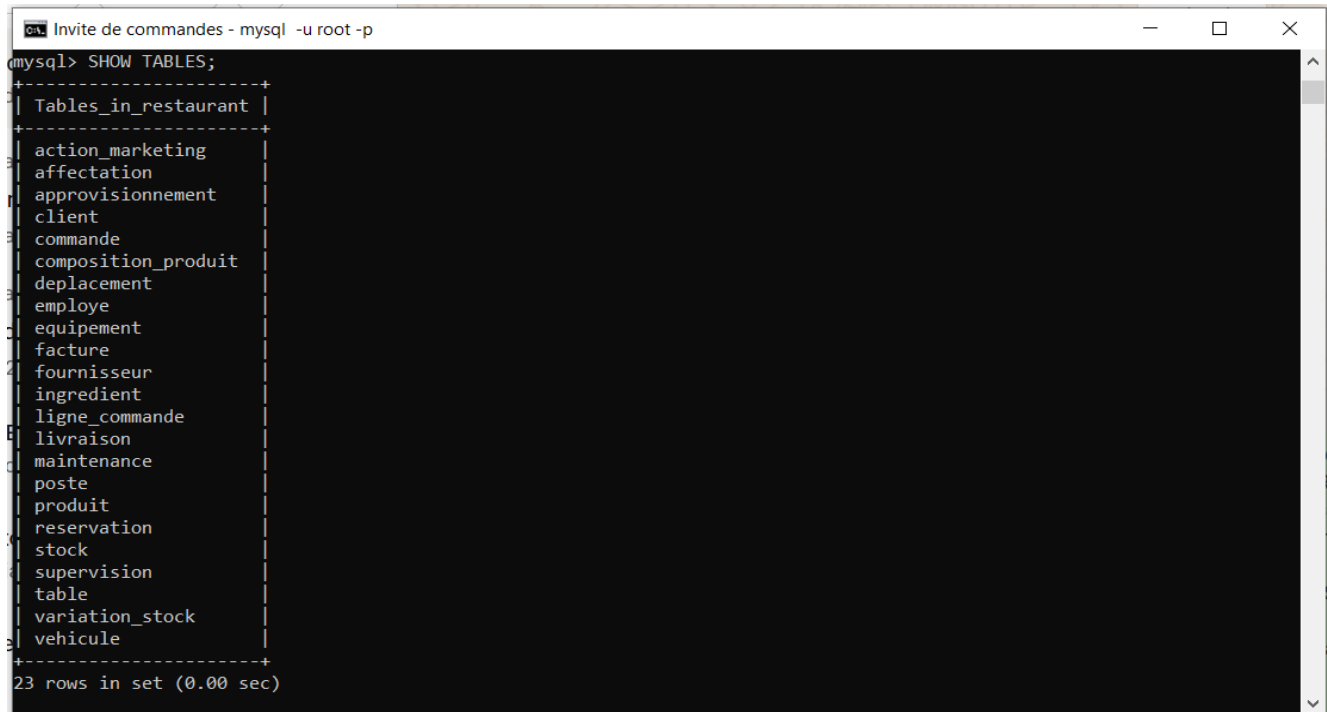
When prompted, we entered the MySQL root password. The absence of any error message confirmed that the import was successful.

6.3 Verifying the Import

After importing, we verified that the database and its 23 tables were correctly present in MySQL by running the following commands:

```
mysql -u root -p
```

```
SHOW DATABASES;  
USE Restaurant;  
SHOW TABLES;
```



The screenshot shows a terminal window titled "Invite de commandes - mysql -u root -p". The prompt is "mysql> SHOW TABLES;". The output is a list of 23 tables enclosed in a dashed box, with the first table "Tables_in_restaurant" highlighted. The tables listed are: action_marketing, affectation, approvisionnement, client, commande, composition_produit, deplacement, employe, equipement, facture, fournisseur, ingredient, ligne_commande, livraison, maintenance, poste, produit, reservation, stock, supervision, table, variation_stock, and vehicule. At the bottom, it says "23 rows in set (0.00 sec)".

```
mysql> SHOW TABLES;  
+-----+  
| Tables_in_restaurant |  
+-----+  
| action_marketing  
| affectation  
| approvisionnement  
| client  
| commande  
| composition_produit  
| deplacement  
| employe  
| equipement  
| facture  
| fournisseur  
| ingredient  
| ligne_commande  
| livraison  
| maintenance  
| poste  
| produit  
| reservation  
| stock  
| supervision  
| table  
| variation_stock  
| vehicule  
+-----+  
23 rows in set (0.00 sec)
```

Step 7: Testing the Django–MySQL Connection

The final step of Phase 1 was to verify that Django communicates correctly with the MySQL database. This check is performed with the following command, run from the root folder of the project:

```
python manage.py check  
  
# Expected result:  
# System check identified no issues (0 silenced).
```

A result with zero errors confirms that the Phase 1 configuration is complete and that the project is ready for Phase 2, which involves writing the Django models for all 23 database tables.

```
C:\Windows\System32\cmd.exe
3      Plantain      12000
1      Poivrons      6500
3      Poivrons      12000
1      Poulet        6500
3      Poulet        12000
4      Riz           9000
6      Riz           4500
2      Sel           8000
4      Sel           9000
6      Sel           4500
1      Sucre         6500
3      Sucre         12000
5      Sucre         9800
1      Tomates       6500
3      Tomates       12000
4      Tomates       9000
6      Tomates       4500
2      Viande de Boeuf 8000
4      Viande de Boeuf 9000
6      Viande de Boeuf 4500
Chiffre_Affaires_Novembre_2025
21500
Nom    Prenom  Tel      Email
Fopa   Franck  687531512  franckfopa875@gmail.com
Djeutoga Nelson  691120602  ndjeutogatofeun@gmail.com
Payne  Success 672506316  successpayne07@gmail.com
Wouaba Brayan  687654144  wouabab@gmail.com

C:\Users\hp\Desktop\PROJECT 1\Restaurant-Management-App>python manage.py check
System check identified no issues (0 silenced).
```

3. Setting Up the GitHub Repository

Alongside the Django project configuration, our group set up the GitHub repository for the project, in compliance with the project requirement that mandates the use of GitHub for source code management and collaborative development.

3.1 Creating GitHub Accounts

Each member of the group created a personal account on github.com. Usernames were chosen to be professional and easily identifiable by the lecturer, as the repository is set to Public.

3.2 Creating the Repository

The group leader created the public repository with the following settings:

Here is the github link to access our repository : <https://github.com/pettangroddysenpai-ship-it/Restaurant-Management-App>

Setting	Value Chosen
Repository Name	Restaurant-Management-App
Visibility	Public

README File	Added automatically
.gitignore	Python template selected
Repository Link	https://github.com/pettangroddysenpai-ship-it/Restaurant-Management-App

1
General

Owner *


pettangroddysenpai-ship-it

/
Repository name *

Restaurant - Management Apps

✔ Your new repository will be created as Restaurant---Management-Apps.
The repository name can only contain ASCII letters, digits, and the characters ., -, and _

Great repository names are short and memorable. How about [refactored-computing-machine?](#)

Description

use the services at a restaurant (Delivery, Orders, Reservation, Transport, Management of employees and users)

233 / 350 characters

2
Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

On

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

Python

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

3.3 Adding Collaborators

The remaining group members were added as collaborators through the Settings → Collaborators page of the repository. Each member received an email invitation from GitHub which they accepted in order to gain push access to the repository.

```
# Navigate directly to the collaborators page:
https://github.com/pettangroddysenpai-ship-it/Restaurant-Management-App/settings/access
```

General

Access

Collaborators

Moderation options

Code and automation

Branches

Tags

Rules

Actions

Models

Webhooks

Copilot

Environments

Codespaces

Pages

Security

Advanced Security

Deploy keys

Secrets and variables

Collaborators and teams

Public repository

Manage visibility

Direct access

Manage access

Add people

Get team access controls and discussions for your contributors in an organization.

Create an organization

3.4 Cloning the Repository and First Push

After accepting their invitations, each group member cloned the repository onto their local machine. The group leader then performed the first push of the Django project structure to GitHub:

Restaurat...

pettangroddysenpai-ship-it

MySQL - Begin Your Download

github.com/pettangroddysenpai-ship-it/Restaurant-Management-App

Importer les marque-...

Débuter avec Firefox

Vocal Remover and Is...

Llama Coder - AI Cod...

Paper2Galgame - Turn...

Recent Files | TurboScr...

Automatic Transcription...

Arduino on Wokwi - O...

pettangroddysenpai-ship-it / Restaurant-Management-App

Code

Issues

Pull requests

Actions

Projects

Wiki

Security

Insights

Settings

Restaurant-Management-App

Public

Pin

Watch 0

Fork 0

Star 0

main

1 Branch

0 Tags

Go to file

Add file

Code

pettangroddysenpai-ship-it Initial commit

b3c9311 · 1 minute ago · 1 Commit

.gitignore

README.md

README

Restaurant-Management-App

Django Restaurant Management Web App Project of PKFIE python course project. Aimed at creating an application in order to ease the services at a restaurant (Delivery, Orders, Reservation, Transport, Management of employees and users)

About

Django Restaurant Management Web App Project of PKFIE python course project. Aimed at creating an application in order to ease the services at a restaurant (Delivery, Orders, Reservation, Transport, Management of employees and users)

Readme

Activity

0 stars

0 watching

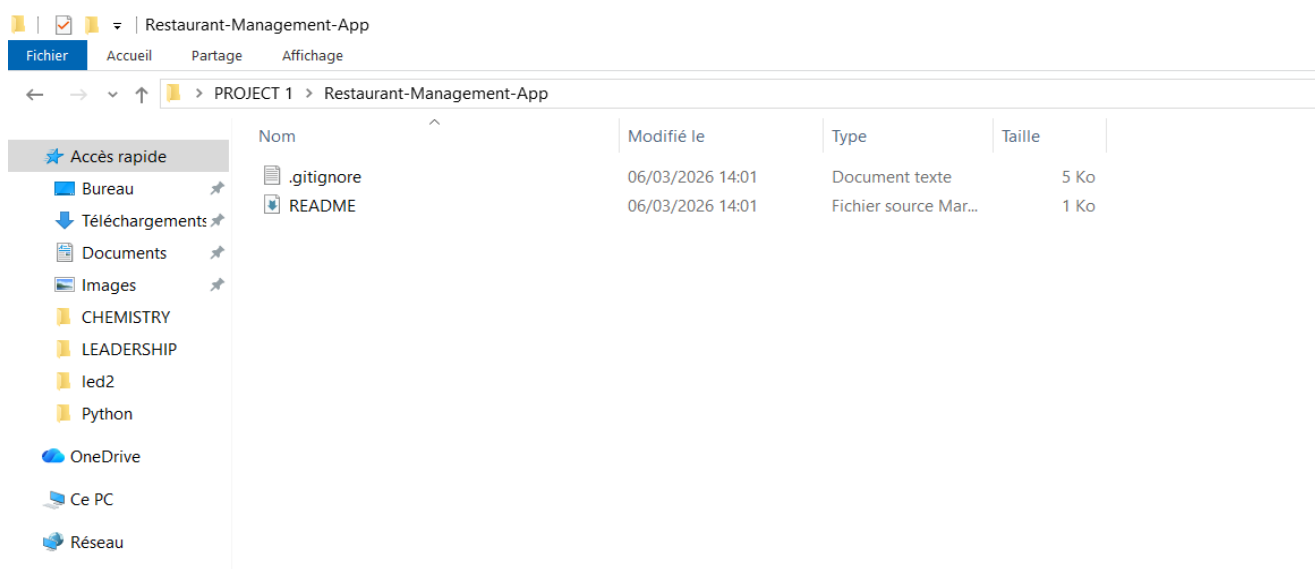
0 forks

Releases

No releases published

Create a new release

Packages



4. Phase 1 Summary

All steps of Phase 1 were completed successfully. The table below provides a full summary of the progress made during this phase:

#	Step	Status	Done By
1	Install required Python packages (Django, mysqlclient, pillow, python-decouple)	✓ Complete	Group
2	Create the Django project with django-admin startproject	✓ Complete	Group
3	Create the 8 Django application modules	✓ Complete	Group
4	Configure MySQL connection in settings.py	✓ Complete	Group
5	Register all 8 apps in INSTALLED_APPS	✓ Complete	Group
6	Install MySQL, add to PATH, and import Base_Restaurant.sql	✓ Complete	Group
7	Test the Django–MySQL connection with manage.py check	✓ Complete	Group
8	Create GitHub repository and add all collaborators	✓ Complete	Group

With Phase 1 fully complete, our group is now ready to begin Phase 2, which will involve writing the Django models for all 23 database tables and implementing the authentication system with role-based access control for the six user types defined in the project specifications.

2. Phase 2 : Django Models (23 SQL Tables in Python)

Phase 2 is the most foundational phase of the project. It involves translating all 23 existing MySQL tables into Python model classes within Django. Since the database tables were already created and populated via the Base_Restaurant.sql file, each model class was configured with `managed = False`, which instructs Django not to attempt to recreate or migrate these tables it simply maps to them.

2.1 Orders App Models

The orders app handles the core transactional data of the restaurant: clients, tables, reservations, orders, and order line items. The following models were defined in `orders/models.py`:

- Client Stores customer identity and contact details.
- TableRestaurant Maps to the SQL 'Table' entity (renamed to avoid conflict with Python's reserved word 'Table').
- Reservation Links a client to a table with date, time, and status.
- Commande Represents a full order with total amount and payment method.
- LigneCommande A junction table for order line items, linking orders to products with quantity and unit price.

2.2 Products App Models

The products app manages recipes, ingredients, and the composition of each dish. Three models were defined in `products/models.py`:

- Ingredient Represents a raw ingredient with its unit of measure and type.
- Produit A dish or product, linked to the employee who created it, with cooking time and serving count.
- CompositionProduit A junction table linking products to their ingredients with quantities used, implementing a composite primary key.

2.3 Inventory, HR, Delivery & Finance Models

The remaining four app modules each map to their respective SQL tables. The table below summarizes each app's models and key implementation notes:

App File	SQL Tables	Key Points
inventory/models.py	Stock, Variation_Stock, Approvisionnement, Fournisseur	Stock uses <code>id_Ingredient</code> as primary key (same as Ingredient PK)
hr/models.py	Employe, Poste, Affectation, Supervision	Affectation has a composite PK: (<code>id_Employe</code> , <code>id_Poste</code>)
delivery/models.py	Vehicule, Deplacement, Livraison	Deplacement has <code>id_Chauffeur</code> as FK to Employe

App File	SQL Tables	Key Points
finance/models.py	Facture, Equipement, Maintenance, Action_Marketing	Facture.Type distinguishes client invoices from supply/salary/marketing records

For models with composite primary keys (such as Affectation), the `unique_together` Meta option was used to enforce uniqueness across the two foreign key fields, as Django does not natively support composite primary keys in the traditional sense.

```

models.py U
C: > Users > hp > Desktop > PROJECT 1 > Restaurant-Management-App > products > models.py > ...
1 from django.db import models
2 # Create your models here.
3 class Ingredient(models.Model):
4     id_ingredient = models.AutoField(primary_key=True, db_column='id_ingredient')
5     nom = models.CharField(max_length=45, db_column='Nom')
6     unite_de_mesure = models.CharField(max_length=45, db_column='Unite_de_Mesure')
7     type = models.CharField(max_length=30, db_column='Type')
8
9     class Meta:
10         managed = False
11         db_table = 'Ingredient'
12         def __str__(self): return self.nom
13
14 class Produit(models.Model):
15     id_produit = models.AutoField(primary_key=True, db_column='id_Produit')
16     id_createur = models.ForeignKey('hr.Employe', on_delete=models.SET_NULL, null=True, db_column='id_Createur')
17     nom = models.CharField(max_length=30, db_column='Nom')
18     description = models.CharField(max_length=45, db_column='Description')
19     duree_cuisson = models.CharField(max_length=20, db_column='Duree_Cuisson')
20     nombre_personnes = models.IntegerField(db_column='Nombre_Personnes')
21
22     class Meta:
23         managed = False
24         db_table = 'Produit'
25         def __str__(self): return self.nom
26
27 class CompositionProduit(models.Model):
28     id_produit = models.ForeignKey(Produit, on_delete=models.CASCADE, db_column='id_Produit')
29     id_ingredient = models.ForeignKey(Ingredient, on_delete=models.CASCADE, db_column='id_Ingredient')
30     quantite_utilisee = models.DecimalField(max_digits=10, decimal_places=2, db_column='Quantite_Utilisee')
31
32     class Meta:
33         managed = False
34         db_table = 'Composition_Produit'
35         unique_together = (('id_produit', 'id_ingredient'))

```

```
models.py U X
C: > Users > hp > Desktop > PROJECT 1 > Restaurant-Management-App > orders > models.py > TableRestaurant > Meta
1 from django.db import models
2
3 # Create your models here.
4 class Client(models.Model):
5     id_client = models.AutoField(primary_key=True, db_column='id_Client')
6     nom = models.CharField(max_length=20, db_column='Nom')
7     prenom = models.CharField(max_length=20, db_column='Prenom')
8     tel = models.CharField(max_length=15, db_column='Tel')
9     email = models.CharField(max_length=45, db_column='Email')
10    type_de_client = models.CharField(max_length=20, db_column='Type_de_Client')
11
12    class Meta:
13        managed = False
14        db_table = 'Client'
15
16    def __str__(self):
17        return f'{self.prenom} {self.nom}'
18
19
20 class TableRestaurant(models.Model): # 'Table' is a reserved word!
21     id_table = models.AutoField(primary_key=True, db_column='id_Table')
22     numero_table = models.CharField(max_length=3, db_column='Numero_Table')
23     capacite = models.IntegerField(db_column='Capacite')
24     commentaire = models.TextField(db_column='Commentaire', blank=True, null=True)
25
26    class Meta:
27        managed = False
28        db_table = 'Table'
29
30    def __str__(self):
31        return f'Table {self.numero_table}'
32
33
34 class Reservation(models.Model):
35     id_reservation = models.AutoField(primary_key=True, db_column='id_Reservation')
36     id_client = models.ForeignKey(Client, on_delete=models.CASCADE, db_column='id_Client')
37     id_table = models.ForeignKey(TableRestaurant, on_delete=models.CASCADE, db_column='id_Table')
38     date = models.DateField(db_column='Date')
39     heure = models.TimeField(db_column='Heure')
40     statut = models.CharField(max_length=45, db_column='Statut')
41
42    class Meta:
43        managed = False
44        db_table = 'Reservation'
45
46
47 class Commande(models.Model):
48     id_commande = models.AutoField(primary_key=True, db_column='id_Commande')
49     id_client = models.ForeignKey(Client, on_delete=models.CASCADE, db_column='id_Client')
50     date = models.DateTimeField(db_column='Date')
51     type = models.CharField(max_length=30, db_column='Type')
52     montant_total = models.IntegerField(db_column='Montant_Total')
53     mode_de_paiement = models.CharField(max_length=45, db_column='Mode_de_Paiement')
54
55    class Meta:
56        managed = False
57        db_table = 'Commande'
58
59    def __str__(self):
60        return f'Commande #{self.id_commande}'
61
62
63 class LigneCommande(models.Model):
64     id_commande = models.ForeignKey(Commande, on_delete=models.CASCADE, db_column='id_Commande')
65     id_produit = models.ForeignKey('products.Produit', on_delete=models.CASCADE, db_column='id_Produit')
66     quantite = models.DecimalField(max_digits=10, decimal_places=2, db_column='Quantite')
67     prix_unitaire = models.IntegerField(db_column='Prix_Unitaire')
68
69    class Meta:
70        managed = False
71        db_table = 'Ligne_Commande'
72        unique_together = [('id_commande', 'id_produit')]
```

3. Phase 3 : Authentication & User Roles

Django provides a built-in authentication system that was extended in this phase to support six distinct user roles corresponding to the restaurant's organizational structure. Rather than using Django's default User model, a custom user model (CustomUser) was created to add role-based access control.

3.1 Custom User Model

The CustomUser class extends Django's AbstractUser and introduces a role field with six predefined choices. It also includes an optional link to the Employe table via an integer field:

Role Key	Display Name
admin	Administrateur
director	Directeur
head_chef	Chef Cuisinier
cook	Cuisinier
server	Serveur
stock_manager	Gestionnaire de Stock

Django has a built-in authentication system. You will extend it to support the 6 roles defined in the project.

Step 11: Create the Custom User Model

In the users app, create a custom user that links to an Employe and has a role field:

```
# users/models.py
from django.contrib.auth.models import AbstractUser
from django.db import models

class CustomUser(AbstractUser):
    ROLES = [
        ('admin', 'Administrateur'),
        ('director', 'Directeur'),
        ('head_chef', 'Chef Cuisinier'),
        ('cook', 'Cuisinier'),
        ('server', 'Serveur'),
        ('stock_manager', 'Gestionnaire de Stock'),
    ]
    role = models.CharField(max_length=20, choices=ROLES, default='server')
    # Optional: link to your Employe table
    employe_id = models.IntegerField(null=True, blank=True)

    def __str__(self):
        return f'{self.username} ({self.get_role_display()})'
```

Then in settings.py, add this line so Django uses your custom user:

```
AUTH_USER_MODEL = 'users.CustomUser'
```

Step 12: Run the First Migration

This creates the users table (and Django's internal tables) in your MySQL database:

```
python manage.py makemigrations users
python manage.py migrate
python manage.py createsuperuser # Create your admin account
```

Step 13: Create Login and Logout Views

Django already has built-in login/logout views. In users/urls.py:

```
# users/urls.py
from django.contrib.auth import views as auth_views
from django.urls import path

urlpatterns = [
    path('login/', auth_views.LoginView.as_view(template_name='users/login.html'), name='login'),
    path('logout/', auth_views.LogoutView.as_view(), name='logout'),
]
```

Step 14: Create Role-Based Permission Decorators

Create a file users/decorators.py that you will use to protect any view by role:

```
# users/decorators.py
from django.contrib.auth.decorators import login_required
from django.core.exceptions import PermissionDenied
from functools import wraps

def role_required(*roles):
    def decorator(view_func):
        @wraps(view_func)
        @login_required
        def _wrapped_view(request, *args, **kwargs):
            if request.user.role in roles:
                return view_func(request, *args, **kwargs)
            raise PermissionDenied
        return _wrapped_view
    return decorator

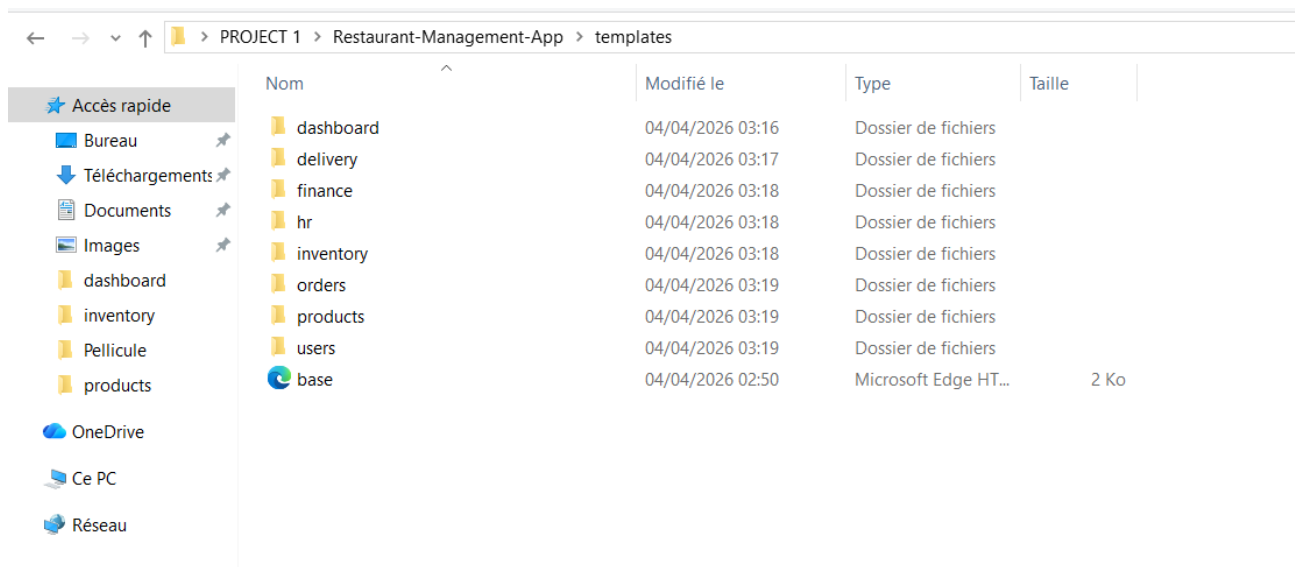
# Usage in any view:
# @role_required('admin', 'director')
# def delete_employee(request, pk): ...
```

4. Phase 4 Templates & Base Interface

Before building any module-specific pages, Phase 4 establishes the shared HTML layout infrastructure that all pages will inherit. This follows Django's template inheritance pattern, where a single base template defines the overall page structure, and child templates override specific content blocks.

4.1 Static Files & Template Folder Structure

Two key configuration settings were added to settings.py: TEMPLATES (to specify the global templates directory) and STATICFILES_DIRS (to locate CSS and JavaScript assets). The following folder structure was created manually at the project root:



The screenshot shows a file explorer window with the path: PROJECT 1 > Restaurant-Management-App > templates. The left sidebar shows a navigation pane with 'Accès rapide' and various folders like 'Bureau', 'Téléchargements', 'Documents', 'Images', 'dashboard', 'inventory', 'Pellicule', 'products', 'OneDrive', 'Ce PC', and 'Réseau'. The main pane displays a table of files and folders within the 'templates' directory.

Nom	Modifié le	Type	Taille
dashboard	04/04/2026 03:16	Dossier de fichiers	
delivery	04/04/2026 03:17	Dossier de fichiers	
finance	04/04/2026 03:18	Dossier de fichiers	
hr	04/04/2026 03:18	Dossier de fichiers	
inventory	04/04/2026 03:18	Dossier de fichiers	
orders	04/04/2026 03:19	Dossier de fichiers	
products	04/04/2026 03:19	Dossier de fichiers	
users	04/04/2026 03:19	Dossier de fichiers	
base	04/04/2026 02:50	Microsoft Edge HT...	2 Ko

Each app has its own subdirectory inside templates/, separating HTML files by module. The static/ directory holds shared CSS and JS files accessible from any template using Django's {% static %} template tag.

4.2 Base Template (base.html)

The base.html file is the master layout for the entire application. It was built using Bootstrap 5, loaded directly from CDN to avoid any local installation. The template defines the following structural blocks :

- **Navigation bar** Displays the app name and links to major modules (Dashboard, Commandes, Stock, RH). The logout button is positioned at the right end.
- **Flash messages block** Automatically renders Django's message framework alerts (success, error, warning) between the navbar and the main content area.
- **Content block** An empty {% block content %} that each child template overrides with its specific page content.
- **Extra JavaScript block** An empty {% block extra_js %} placed at the bottom, allowing child templates to add page-specific scripts without polluting the global head.

```

1 <!-- templates/base.html -->
2 <!DOCTYPE html>
3 <html lang='fr'>
4 <head>
5   <meta charset='UTF-8'>
6   <title>{% block title %}Restaurant{% endblock %}</title>
7   <link href='https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css' rel='stylesheet'>
8 </head>
9 <body>
10  <nav class='navbar navbar-dark bg-dark'>
11    <div class='container-fluid'>
12      <span class='navbar-brand'>Restaurant Manager</span>
13      <ul class='navbar-nav flex-row gap-3'>
14        <li><a class='nav-link text-white' href='{% url "dashboard" %}'>Dashboard</a></li>
15        <li><a class='nav-link text-white' href='{% url "orders:list" %}'>Commandes</a></li>
16        <li><a class='nav-link text-white' href='{% url "inventory:list" %}'>Stock</a></li>
17        <li><a class='nav-link text-white' href='{% url "hr:list" %}'>RH</a></li>
18      </ul>
19      <a href='{% url "logout" %}' class='btn btn-outline-light btn-sm'>Déconnexion</a>
20    </div>
21  </nav>
22  <div class='container mt-4'>
23    {% if messages %}
24      {% for message in messages %}
25        <div class='alert alert-{{ message.tags }}'>{{ message }}</div>
26      {% endfor %}
27    {% endif %}
28    {% block content %}{% endblock %}
29  </div>
30  <script src='https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js'></script>
31  {% block extra_js %}{% endblock %}
32 </body>
33 </html>
34

```

4.3 Child Template Pattern

Every other HTML page in the application starts with `{% extends 'base.html' %}` and overrides the `{% block content %}` and optionally `{% block title %}`. This ensures visual consistency across all pages while keeping each template focused only on its specific content.

As a demonstration, the `orders/list.html` template extends `base.html` and renders a Bootstrap-styled table of all orders, with columns for ID, client, date, total amount, and an action button linking to the order detail view.

```

1 <!-- templates/orders/list.html -->
2 {% extends 'base.html' %}
3 {% block title %}Liste des Commandes{% endblock %}
4 {% block content %}
5   <h2>Commandes</h2>
6   <table class='table table-striped'>
7     <thead>
8       <tr><th>ID</th><th>Client</th><th>Date</th><th>Montant</th><th>Actions</th></tr>
9     </thead>
10    <tbody>
11      {% for commande in commandes %}
12        <tr>
13          <td>{{ commande.id_commande }}</td>
14          <td>{{ commande.id_client }}</td>
15          <td>{{ commande.date }}</td>
16          <td>{{ commande.montant_total }} FCF&lt;td>
17            <a href='{% url "orders:detail" commande.id_commande %}' class='btn btn-sm btn-info'>Voir</a></td>
18          </tr>
19        {% endfor %}
20      </tbody>
21    </table>
22  {% endblock %}
23

```

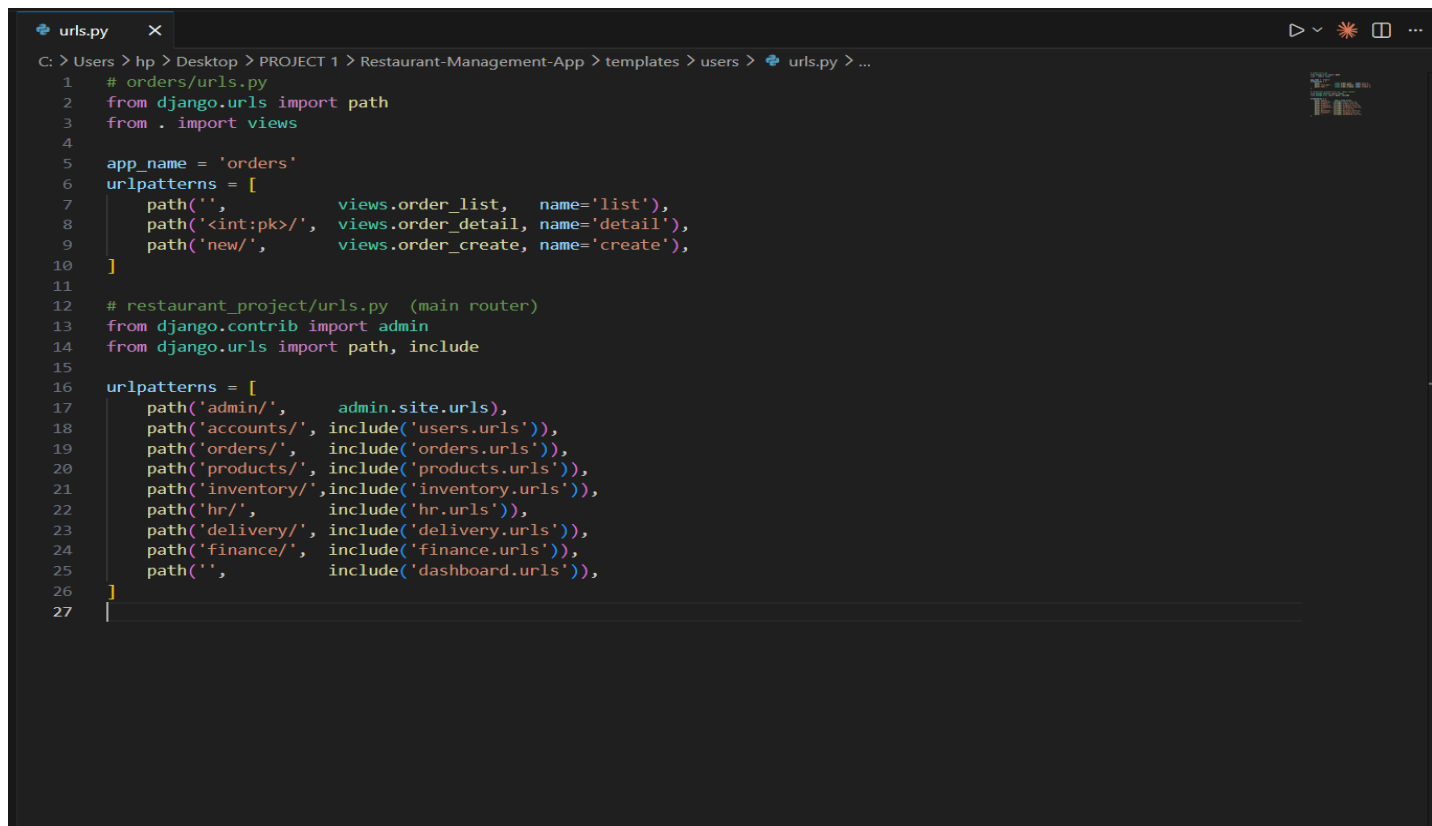
5. Phase 5 Views, URLs & Business Logic

Phase 5 is where the application becomes fully functional. Each module receives its views (the Python functions that process HTTP requests), URL patterns (the routes that map URLs to views), and business logic (the rules that govern data processing). Every module follows the same structural pattern: list, detail, create, edit, and delete.

5.1 Standard CRUD Pattern Orders Module

The Orders module serves as the reference implementation for the CRUD pattern applied across all modules. It involves four key views and an auxiliary helper function:

1. `order_list` Retrieves all orders sorted by date (most recent first) and renders them in the list template.
2. `order_detail` Retrieves a single order by primary key and its associated line items, then renders the detail view.
3. `order_create` Handles both GET (display the creation form) and POST (process form submission). On POST, it creates the `Commande` record, iterates over submitted line items to create `LigneCommande` records, calculates the total amount, and triggers automatic stock deduction.
4. `deduct_stock_for_product` A helper function called inside `order_create` that retrieves the product's ingredient composition and reduces each ingredient's stock quantity accordingly. It also logs a 'sortie' entry in the `VariationStock` table for traceability.



```
urls.py
C: > Users > hp > Desktop > PROJECT 1 > Restaurant-Management-App > templates > users > urls.py > ...
1 # orders/urls.py
2 from django.urls import path
3 from . import views
4
5 app_name = 'orders'
6 urlpatterns = [
7     path('', views.order_list, name='list'),
8     path('<int:pk>/', views.order_detail, name='detail'),
9     path('new/', views.order_create, name='create'),
10 ]
11
12 # restaurant_project/urls.py (main router)
13 from django.contrib import admin
14 from django.urls import path, include
15
16 urlpatterns = [
17     path('admin/', admin.site.urls),
18     path('accounts/', include('users.urls')),
19     path('orders/', include('orders.urls')),
20     path('products/', include('products.urls')),
21     path('inventory/', include('inventory.urls')),
22     path('hr/', include('hr.urls')),
23     path('delivery/', include('delivery.urls')),
24     path('finance/', include('finance.urls')),
25     path('', include('dashboard.urls')),
26 ]
27
```

```
views.py 2, U X
C: > Users > hp > Desktop > PROJECT 1 > Restaurant-Management-App > templates > dashboard > views.py > ...
1  # Create your views here.
2  from django.shortcuts import render, get_object_or_404, redirect
3  from django.contrib.auth.decorators import login_required
4  from django.contrib import messages
5  from django.db import transaction
6  from .models import Commande, LigneCommande, Client
7  from inventory.models import Stock, VariationStock
8  from products.models import CompositionProduit
9  import datetime
10
11  Windsurf: Refactor | Explain | Generate Docstring | X
12  @login_required
13  def order_list(request):
14      commandes = Commande.objects.all().order_by('-date')
15      return render(request, 'orders/list.html', {'commandes': commandes})
16
17  Windsurf: Refactor | Explain | Generate Docstring | X
18  @login_required
19  def order_detail(request, pk):
20      commande = get_object_or_404(Commande, pk=pk)
21      lignes = LigneCommande.objects.filter(id_commande=commande)
22      return render(request, 'orders/detail.html', {'commande': commande, 'lignes': lignes})
23
24  Windsurf: Refactor | Explain | Generate Docstring | X
25  @login_required
26  def order_create(request):
27      if request.method == 'POST':
28          # 1. Create the order
29          commande = Commande.objects.create(
30              id_client_id=request.POST['client_id'],
31              date=datetime.datetime.now(),
32              type=request.POST['type'],
33              montant_total=0,
34              mode_de_paiement=request.POST['paiement'],
35          )
36          total = 0
37          # 2. Add each line item
38          for produit_id, qty in zip(request.POST.getlist('produit_id'), request.POST.getlist('qty')):
39              prix = int(request.POST.get(f'prix_{produit_id}', 0))
40              ligne_commande = LigneCommande.objects.create(
41                  id_commande=commande, id_produit_id=produit_id,
42                  quantite=qty, prix_unitaire=prix
43              )
44              total += prix * float(qty)
45          # 3. Deduct ingredients from Stock automatically
46          deduct_stock_for_product(produit_id, float(qty))
47          # 4. Update total
48          commande.montant_total = int(total)
49          commande.save()
50          messages.success(request, 'Commande créée avec succès!')
51          return redirect('orders:detail', pk=commande.pk)
52
53      clients = Client.objects.all()
54      return render(request, 'orders/create.html', {'clients': clients})
55
56  Windsurf: Refactor | Explain | X
57  def deduct_stock_for_product(produit_id, qty_ordered):
58      '''Automatically reduce stock when an order is placed'''
59      compositions = CompositionProduit.objects.filter(id_produit_id=produit_id)
60      for comp in compositions:
61          stock = Stock.objects.get(id_ingredient=comp.id_ingredient)
62          qty_used = comp.quantite_utilisee * qty_ordered
63          stock.quantite_actuelle -= qty_used
64          stock.save()
65          # Log the variation
66          variation_stock = VariationStock.objects.create(
67              id_ingredient=comp.id_ingredient,
68              date=datetime.date.today(),
69              type='sortie',
70              quantite=qty_used
71          )
```

5.2 URL Wiring

Each app defines its own `urls.py` file with an `app_name` namespace to avoid URL name conflicts across modules. These app-level URL files are then included in the main `restaurant_project/urls.py` file using Django's `include()` function.

The main URL configuration registers the following route prefixes:

URL Prefix	App Included	Purpose
admin/	django.contrib.admin	Django Admin Panel
accounts/	users.urls	Login / Logout
orders/	orders.urls	Order Management
products/	products.urls	Product & Recipe Management
inventory/	inventory.urls	Stock & Inventory
hr/	hr.urls	Human Resources
delivery/	delivery.urls	Delivery Tracking
finance/	finance.urls	Finance & Invoices
/ (root)	dashboard.urls	Dashboard Home Page

```
views.py C:\...\inventory X views.py C:\...\finance ●
C: > Users > hp > Desktop > PROJECT 1 > Restaurant-Management-App > templates > inventory > views.py
1  # Create your views here.
2  from django.shortcuts import render, get_object_or_404, redirect
3  from django.contrib.auth.decorators import login_required
4  from django.contrib import messages
5  from django.db import transaction
6  from .models import Commande, LigneCommande, Client
7  from inventory.models import Stock, VariationStock
8  from products.models import CompositionProduit
9  from django.db import models as django_models
10 from .models import Stock
11 from products.models import Ingredient
12 import datetime
13
14 @login_required
15 def stock_alerts(request):
16     '''Returns ingredients where current quantity <= alert threshold'''
17     low_stock = Stock.objects.filter(
18         quantite_actuelle_lte=django_models.F('seuil_alerte')
19     ).select_related('id_ingredient')
20     return render(request, 'inventory/alerts.html', {'low_stock': low_stock})
21
22 @login_required
23 def order_list(request):
24     commandes = Commande.objects.all().order_by('-date')
25     return render(request, 'orders/list.html', {'commandes': commandes})
26
27 @login_required
28 def order_detail(request, pk):
29     commande = get_object_or_404(Commande, pk=pk)
30     lignes = LigneCommande.objects.filter(id_commande=commande)
31     return render(request, 'orders/detail.html', {'commande': commande, 'lignes': lignes})
32
```

5.3 Low-Stock Alert Logic (Inventory Module)

A dedicated view was created in `inventory/views.py` to detect ingredients whose current quantity is at or below their defined alert threshold. This uses Django's ORM `F()` expression to compare two fields within the same model row, avoiding the need for raw SQL.

The query filters `Stock` objects where `quantite_actuelle` is less than or equal to `seuil_alerte`. The `select_related()` call ensures the related `Ingredient` data is fetched in the same database query (JOIN), avoiding N+1 query issues.

This view powers the stock alert widget displayed on the dashboard, providing real-time visibility into critical inventory levels.

5.4 Invoice Generation (Finance Module)

A printable invoice view was implemented in `finance/views.py`. This view retrieves a `Commande` object by its primary key and its associated `LigneCommande` records, then passes them to a dedicated invoice template. The template includes a 'Print' button that triggers the browser's native print dialog via a JavaScript onclick event.

This approach avoids any external PDF library dependency while providing a functional and printable invoice output that can also be saved as PDF from the browser's print dialog.

```
views.py 3
C: > Users > hp > Desktop > PROJECT 1 > Restaurant-Management-App > templates > finance > views.py > ...
1  # Create your views here.
2  from django.shortcuts import render, get_object_or_404, redirect
3  from django.contrib.auth.decorators import login_required
4  from django.contrib import messages
5  from django.db import transaction
6  from .models import Commande, LigneCommande, Client
7  from inventory.models import Stock, VariationStock
8  from products.models import CompositionProduit
9  from django.shortcuts import render, get_object_or_404
10 from orders.models import Commande, LigneCommande
11 import datetime
12
Windsurf: Refactor | Explain | Generate Docstring | X
13 @login_required
14 def generate_invoice(request, commande_id):
15     commande = get_object_or_404(Commande, pk=commande_id)
16     lignes = LigneCommande.objects.filter(id_commande=commande)
17     return render(request, 'finance/invoice.html', {
18         'commande': commande,
19         'lignes': lignes,
20     })
21     # In finance/invoice.html, add a 'Print' button:
22     # <button onclick='window.print()'>Imprimer la Facture</button>
23
Windsurf: Refactor | Explain | Generate Docstring | X
24 @login_required
25 def order_list(request):
26     commandes = Commande.objects.all().order_by('-date')
27     return render(request, 'orders/list.html', {'commandes': commandes})
28
Windsurf: Refactor | Explain | Generate Docstring | X
29 @login_required
30 def order_detail(request, pk):
31     commande = get_object_or_404(Commande, pk=pk)
32     lignes = LigneCommande.objects.filter(id_commande=commande)
33     return render(request, 'orders/detail.html', {'commande': commande, 'lignes': lignes})
```

7. Challenges Encountered & Solutions

Several technical and organizational challenges arose during the development of this project. The following describes the most significant ones and the approaches used to resolve them.

Challenge 1: Composite Primary Keys in Django

Django does not natively support composite primary keys. However, several of the existing SQL tables (such as `Affectation`, `LigneCommande`, and `CompositionProduit`) used composite primary keys made of two foreign key columns.

Solution: The `unique_together` Meta option was applied in each affected model to enforce the uniqueness constraint at the Django ORM level. This approach successfully mirrors the composite PK behavior of the SQL tables without modifying the database schema.

Challenge 2: Reserved Word Conflict ('Table')

One of the SQL tables is named 'Table', which is a reserved keyword in Python and in Django's ORM. Naming a model class 'Table' caused syntax conflicts and unexpected behavior.

Solution: The model class was renamed to `TableRestaurant` in Python, while the `db_table` Meta attribute was set to 'Table' to maintain the correct mapping to the existing SQL table. This resolved the conflict without altering the database.

Challenge 3: AUTH_USER_MODEL Applied After Initial Migration

During early testing, the custom user model setting (`AUTH_USER_MODEL = 'users.CustomUser'`) was added to `settings.py` after initial migrations had already been run. This caused migration dependency errors and broken foreign key references throughout Django's internal tables.

Solution: The database was reset entirely by dropping and recreating the Django-managed tables, and all migrations were re-run from scratch. This highlighted the critical importance of defining the custom user model before any migration is executed a step now documented as a warning in the development guide.

8. Conclusion

Phase 2 established a solid data foundation by mapping all SQL tables to Django model classes, respecting the existing schema through the `managed = False` configuration. Phase 3 introduced a secure, role-based access control system that differentiates between six types of users, ensuring that each staff member can only access the features relevant to their role. Phase 4 delivered a coherent and reusable user interface built on Django's template inheritance system, providing a consistent experience across all modules. Phase 5 brought the application to life with CRUD views, URL routing, and key business logic including automatic stock deduction and invoice generation.

From a project management perspective, the Gantt chart enabled clear scheduling and prioritization of tasks, while the class diagrams provided a shared visual reference for the system's data architecture. These tools were invaluable in coordinating development across multiple modules.